

PARALELNI SISTEMI

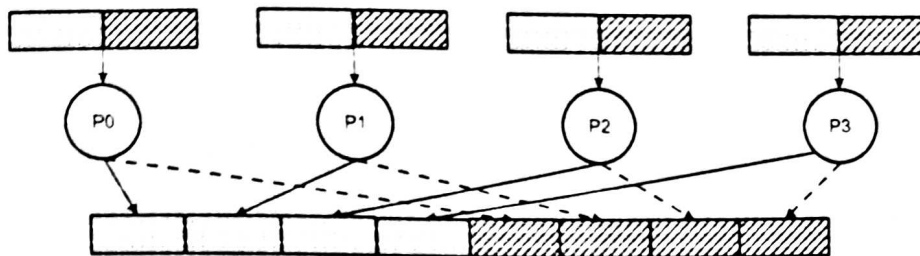
1. Neka je dat CUDA kernel i odgovarajuća funkcija koja ga poziva.

```
__global__ void vecAddKernel(float* A_d, float* B_d, float* C_d, int n)
{
    int i = threadIdx.x + blockDim.x * blockIdx.x;
    if(i < n)
        C_d[i] = A_d[i] + B_d[i];
}

int vectAdd(float* A, float* B, float* C, int n) //podrazumevati da su nizovi A, B i C dužine n
{
    int size = n * sizeof(float);
    cudaMalloc((void **) &A_d, size);
    cudaMalloc((void **) &B_d, size);
    cudaMalloc((void **) &C_d, size);
    cudaMemcpy(A_d, A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(B_d, B, size, cudaMemcpyHostToDevice);
    vecAddKernel<<<ceil(n/256), 256>>>>(A_d, B_d, C_d, n);
    cudaMemcpy(C, C_d, size, cudaMemcpyDeviceToHost);
}
```

- Ako su A,B,C nizovi od 1000 elemenata, koliko će blokova biti pokrenuto?
 - Ako su A,B,C nizovi od 1000 elemenata, koliko će warp-ova biti u svakom od blokova?
 - Ako su A,B,C nizovi od 1000 elemenata, koliko će niti biti u gridu?
 - Ako su A,B,C nizovi od 1000 elemenata, da li će postojati divergentnost u izvršenju kernel? Ako da, u kojoj liniji. Objasniti zašto.
 - Ako su A,B,C nizovi od 768 elemenata, da li će postojati divergentnost u izvršenju kernela? Ako da, u kojoj liniji. Objasniti zašto.
 - Šta dati kernel radi?
 - Modifikovati kernel tako da svaki 256. element rezultujućeg niza sadrži srednju vrednost prethodnih 255 elemenata.
- 2.
- Napisati MPI program koji realizuje množenje matrice $A_{n \times n}$ i vektora $B_{n \times 1}$, čime se dobija rezultujuća matrica C. Program pronalazi i prikazuje maksimalni element u rezultujućoj matrici, kao i identifikator procesa koji ga sadrži. Matrice A i B se inicijalizuju u master procesu. Matrica A je podeljena u blokove od po k vrsta (k je zadata konstanta i n je deljivo sa k), a matrica B podeljena u blokove od po k kolona. Master process distribuira odgovarajuće blokove matrice A i odgovarajuće blokove matrice B procesima radnicima. Ukupan broj procesa je $(n/k)^2$. Svaki proces obavlja odgovarajuća izračunavanja i učestvuje u generisanju rezultata koji se prikazuje u master procesu. Predvideti da se slanje blokova od po k vrsta matrice A i od po k kolona matrice B svakom procesu obavlja sa po jednom naredbom MPI_Send kojom se šalje samo 1 izvedeni tip podatka. Za ostatak izračunavanja koristiti grupne operacije.
 - Napisati MPI program kojim se podaci o nizu studenata koji se unose sa tastature u master procesu, prosleđuju odjednom svim procesima u komunikatoru MPI_COMM_WORLD. Za svakog studenta se pamte: broj indeksa(int), ime (string), prezime (string), prosečna ocena(float) i godina studija(int). Zadatak rešiti korišćenjem grupnih operacija i izvedenih tipova podataka.

3. Datoteka *file1.dat* sadrži ukupno 10MB prikupljenih zapisa o mrežnom saobraćaju. Napisati MPI program koji vrši obradu ovih podataka i priprema ih za vizuelizaciju, ujedno vršeći paralelni upis i čitanje datoteke. Na početku, svi procesi vrše čitanje iste količine podataka tako da se pozicija sa koje svaki pojedinačan proces čita podatke ne može predvideti. Pročitane podatke procesi upisuju u datoteku *file2.dat*, tako što ih dele na dva jednaka dela i upisuju u različite blokove, kao na slici (za slučaj od 4 procesa):



Obratiti pažnju na efikasnost paralelizacije upisa.

4. Napisati OpenMP kod koji sadrži sledeće ugnježdene petlje:

```
x = 0;

for (i = 2; i < N; i++)
  for (j = 0; j < M; j++)
  {
    x = x + a[i][j];
    b[i][j] = 4*b[i-2][j];
  }
```

i proučiti da li je moguće izvršiti njihovu paralelizaciju. Ako nije, transformisati petlje tako da paralelizacija bude moguća. Nakon petlji treba prikazati vrednosti za promenljivu x i matricu b, koje su generisane u okviru petlje. Testiranjem sekvencijalnog i paralelnog rešenja za proizvoljno N i M, i proizvoljan broj niti, pokazati korektnost paralelizovanog koda.